VibeCoding Lab 4 — Mastermind Game

AI-Assisted Coding, Debugging and Testing Report

SRN:PES2UG24CS580

Name:Venkatesh

Project: Mastermind Terminal Game
Repository: https://github.com/SETAPESU26/20_mastermind
Purpose: Fix existing bugs and implement the features described in the project README using AI coding assistance.

1. Bug Identification and Fix

Problem identified: The original logic.py incorrectly calculated partial matches when the secret code or guess contained repeated symbols. The same occurrence in the secret code could be counted more than once.

Original faulty code:

```
exact = sum(a == b for a, b in zip(code, guess))

partial = sum(ch in code for ch in guess) - exact

return exact, partial
```

AI-assisted solution: The feedback function was updated to count exact matches first, remove the matched positions from further consideration, and use Counter to track the remaining occurrences of each symbol.

Updated implementation in logic.py:

```
from collections import Counter



def feedback(code, guess):
    exact = sum(a == b for a, b in zip(code, guess))

    unmatched_code = [
        code[i] for i in range(len(code))
        if code[i] != guess[i]
    ]
    unmatched_guess = [
        guess[i] for i in range(len(guess))
        if code[i] != guess[i]
    ]

    code_counts = Counter(unmatched_code)
    partial = 0

    for symbol in unmatched_guess:
        if code_counts[symbol] > 0:
            partial += 1
            code_counts[symbol] -= 1

    return exact, partial
```

Verification: For the secret code 1123 and guess 1112, the original implementation returned (2, 2), while the corrected implementation returned (2, 1). This confirmed that repeated symbols were no longer double-counted.

2. Features Implemented

The existing project was extended while retaining its original file structure and terminal-based gameplay.

- **Difficulty selection:** Added Easy, Medium, and Hard modes.
- **Guess history:** Added the h command to display accepted guesses and their feedback.
- **Input validation:** Invalid guesses are rejected without consuming a turn.
- **Win handling:** The game displays a success message and stops when the code is cracked.
- **Loss handling:** The game reveals the secret code when all turns are used.
- **Quit handling:** The q command allows the player to exit.
- **Game state:** The game stops accepting guesses after completion.

Difficulty configuration:

Mode	Code length	Allowed symbols	Turns
Easy	3	1–4	10
Medium	4	1–6	10
Hard	5	1–6	8

The existing main.py entry point was checked and retained:

```
from game import Mastermind


if __name__ == "__main__":
    Mastermind().run()
```

3. Testing and Results

The following tests were performed during development.

Test	Observed result
Duplicate-symbol feedback	Passed: (2, 1)
All exact matches	Passed: (4, 0)
No matching symbols	Passed: (0, 0)
All partial matches	Passed: (0, 4)
Invalid-length guesses	Rejected without consuming turns
Difficulty selection	Easy, Medium and Hard modes tested
Guess history	Accepted guesses and feedback displayed
Winning the game	Success message displayed and game stopped
Losing the game	Secret code and guess history displayed after turns ran out
Quitting	Game ended and secret code was displayed

4. Conclusion

AI assistance was used to identify and correct the duplicate-counting bug, extend the existing game with the requested features, and guide testing. The updated game was run in VS Code, and the main feedback, difficulty, input-validation, history, win, loss, and quit behaviors were tested.

The final submission includes the updated Python source files, a before video, an after video, and this focused report of the relevant AI-assisted coding work.

Note: This document is a focused summary of the AI-assisted work, not a verbatim transcript of the complete conversation. If the assignment requires the full chat history or a shareable chat link, include that separately.